



## Worksheet 5

**Student Name:** Aryan  
**Branch:** MCA (AI&ML)  
**Semester:** 2nd  
**Subject Name:** DBMS LAB

**UID:** 25MCI10082  
**Section/Group:** 1/A  
**Date of Performance:** 01/03/2026  
**Subject Code:** 25CAP-652

### 1. Aim of the Session

To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic. **(Company Tags: Infosys, Wipro, TCS, Capgemini)**

### 2. Software Requirements

- PostgreSQL (Database Server)
- pgAdmin
- Windows Operating System

### 3. Objective of the Session

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

## 4. Practical / Experiment Steps

### Prerequisite Understanding

- Students should first understand that cursors are used inside PL/pgSQL blocks for row-by-row processing, not in normal set-based SQL queries.
- Students should understand the difference between set-oriented SQL execution and procedural rowlevel execution.
- Students should create a table containing multiple records so that sequential row processing using cursors can be demonstrated.
- The table should contain:
  - A unique identifier (Primary Key)
  - A descriptive attribute (such as employee name or department)
  - A numeric attribute (such as salary or experience) to apply conditional logic
- Students should understand the lifecycle of a cursor, including:
  - DECLARE – defining the cursor query
  - OPEN – executing the cursor
  - FETCH – retrieving rows sequentially
  - CLOSE – releasing the result set
- Students should be familiar with basic IF–ELSE logic for applying business rules during row processing.

## 5. Procedure of the Practical

(i) Start the system and log in to the computer.

(ii) Open PostgreSQL software.

iii) Create and select the database.

Create database Practical5;

---

**(iv) Create table using DDL command.**

```
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY,  
    emp_name VARCHAR(100),  
    salary NUMERIC(10,2),  
    experience INT );
```

---

**(v) Insert records into the table.**

```
INSERT INTO employees(emp_name,experience,salary) VALUES  
( 'Amit',3,50000),  
( 'Rohan',5,70000),  
( 'Isha',2,45000),  
( 'Sneha',4,60000),  
( 'Anish',3,52000);
```

---

**(vi) Display all records.**

```
select * from employees;
```

|   | emp_id<br>[PK] integer  | emp_name<br>character varying (100)  | salary<br>numeric (10,2)  | experience<br>integer  |
|---|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1 | 1                                                                                                          | Amit                                                                                                                    | 50000.00                                                                                                     | 3                                                                                                           |
| 2 | 2                                                                                                          | Rohan                                                                                                                   | 70000.00                                                                                                     | 5                                                                                                           |
| 3 | 3                                                                                                          | Isha                                                                                                                    | 45000.00                                                                                                     | 2                                                                                                           |
| 4 | 4                                                                                                          | Sneha                                                                                                                   | 60000.00                                                                                                     | 4                                                                                                           |
| 5 | 5                                                                                                          | Anish                                                                                                                   | 52000.00                                                                                                     | 3                                                                                                           |

---

## Step 1: Implementing a Simple Forward-Only Cursor

```
DO $$
    DECLARE
        emp_rec record;
        emp_cursor cursor for
            SELECT emp_id, emp_name, salary FROM employees;
    BEGIN
        OPEN emp_cursor;

        LOOP
            FETCH emp_cursor INTO emp_rec;
            EXIT WHEN NOT FOUND;

            RAISE NOTICE 'ID: %, Name: %, Salary: %',
emp_rec.emp_id, emp_rec.emp_name, emp_rec.salary;
        END LOOP;
        CLOSE emp_cursor;
    END $$;
```

```
NOTICE:  ID: 1, Name: Amit, Salary: 50000.00
NOTICE:  ID: 2, Name: Rohan, Salary: 70000.00
NOTICE:  ID: 3, Name: Isha, Salary: 45000.00
NOTICE:  ID: 4, Name: Sneha, Salary: 60000.00
NOTICE:  ID: 5, Name: Anish, Salary: 52000.00
DO
```

```
Query returned successfully in 57 msec.
```

## Step 2: Complex Row-by-Row Manipulation

```
do $$
declare
    emp_rec record;
    emp_cursor cursor for
        select emp_id, experience, salary from employees;
begin
    open emp_cursor;

    loop
        fetch emp_cursor into emp_rec;
        exit when not found;

        if emp_rec.experience >= 4 then
            update employees
            set salary = salary * 1.35
            where emp_id = emp_rec.emp_id;

        elseif emp_rec.experience >= 3 then
            update employees
            set salary = salary * 1.20
            where emp_id = emp_rec.emp_id;

        else
            update employees
            set salary = salary * 1.10
            where emp_id = emp_rec.emp_id;
        end if;

    end loop;

    close emp_cursor;
end $$;
```

|   | emp_id<br>[PK] integer | emp_name<br>character varying (100) | salary<br>numeric (10,2) | experience<br>integer |
|---|------------------------|-------------------------------------|--------------------------|-----------------------|
| 1 | 1                      | Amit                                | 60000.00                 | 3                     |
| 2 | 2                      | Rohan                               | 94500.00                 | 5                     |
| 3 | 3                      | Isha                                | 49500.00                 | 2                     |
| 4 | 4                      | Sneha                               | 81000.00                 | 4                     |
| 5 | 5                      | Anish                               | 62400.00                 | 3                     |

### Step3: Exception and Status Handling

do \$\$

declare

emp\_rec record;

emp\_cursor cursor for

select \* from employees where experience > 10;

begin

open emp\_cursor;

fetch emp\_cursor into emp\_rec;

if not found then

raise notice 'No employees found with experience > 10.';

else

raise notice 'Employees exist.';

end if;

```
close emp_cursor;  
end $$;
```

```
NOTICE: No employees found with experience > 10.  
DO  
  
Query returned successfully in 52 msec.
```

## 6. I/O Analysis (Input / Output) Input:

- Employee records inserted into a database table for row-by-row processing
- PL/pgSQL DO blocks containing cursor logic
- DECLARE, OPEN, FETCH, and CLOSE cursor statements
- Conditional IF–ELSE logic applied within cursor loops
- UPDATE statements executed based on experience or salary conditions
- Exception handling logic for empty result sets **Output:**

- Sequential row-by-row processing of employee records using cursor mechanism
- Display of employee details using FETCH operation
- Conditional salary updates based on defined business rules
- Notice messages generated during cursor execution
- Proper termination of cursor after processing all records
- Successful handling of empty result sets without runtime errors
- Correct execution of procedural SQL demonstrating cursor lifecycle

Screenshots of execution and obtained results are attached.

## 7. Learning Outcomes

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve realworld scenarios like multi-level payroll adjustments or data migrations.